

CODING FOR KIDS

Learn to Think Like a Programmer (Ages 8-14)

15 Chapters * No Computer Needed * Puzzles * Activities

By Daniel Tesfamariam

CODING FOR KIDS: Learn to Think Like a Programmer

Written by Daniel Tesfamariam

Copyright 2025. All rights reserved.

Published for Amazon KDP

Paperback Edition - 8.5 x 11 inches

TABLE OF CONTENTS

Chapter 1: What Is Coding?

Chapter 2: Algorithms in Daily Life

Chapter 3: Sequences & Order

Chapter 4: Loops: Repeat!

Chapter 5: Conditionals: If/Then

Chapter 6: Variables: Containers for Info

Chapter 7: Debugging: Finding Mistakes

Chapter 8: Functions: Mini-Programs

Chapter 9: Input & Output

Chapter 10: Simple Game Design Concepts

Chapter 11: Web Basics: HTML Intro

Chapter 12: App Thinking

Chapter 13: AI Basics for Kids

Chapter 14: Coding Careers

Chapter 15: Your Coding Journey Plan

WELCOME, FUTURE CODER!

Did you know that coding is the most in-demand skill in the world right now? And the best part is: you do NOT need a computer to start learning how to THINK like a programmer! This book teaches you computational thinking - the logical, creative problem-solving approach that all coders use. Each chapter builds on the last, with real-world examples, hands-on activities (no screen required!), and fun challenges. By the end, you will understand how programs work and be ready to start writing your own code!

CHAPTER 1

What Is Coding?

Coding is giving instructions to a computer in a language it understands. Computers are super fast but not smart on their own - they need YOU to tell them exactly what to do! Every app on your phone, every video game, every website was built by someone writing code. Coding is like learning a superpower: once you know it, you can create almost anything you imagine. The best part? Anyone can learn, no matter how old you are.

KEY CONCEPT:

Think of coding like writing a recipe. If you leave out a step or put steps in the wrong order, the recipe fails. Computers follow instructions perfectly - but only if those instructions are perfect too!

CHAPTER 1 ACTIVITIES

Challenge 1:

Write step-by-step instructions for making a sandwich - be VERY specific

Challenge 2:

Have a friend follow your instructions EXACTLY - what went wrong?

Challenge 3:

List 5 things in your life that use computer code

CODE THIS! (Pseudocode Exercise)

Write step-by-step instructions to sort 5 numbers from smallest to largest.

CHAPTER 1 PUZZLE

WORD SEARCH: Find these coding words: ALGORITHM, CODE, BUG, LOOP, VARIABLE, FUNCTION, INPUT, OUTPUT, DEBUG, PROGRAM

[Work Space]

CHAPTER 2

Algorithms in Daily Life

An algorithm is just a step-by-step set of instructions to solve a problem or complete a task. You already use algorithms every day without knowing it! Getting dressed is an algorithm: underwear first, then pants, then shirt, then shoes. Brushing your teeth is an algorithm. Even your morning routine is a series of steps in a specific order. Programmers write algorithms to tell computers how to solve problems - from simple math to driving cars.

KEY CONCEPT:

A good algorithm is: 1) Clear (no confusion), 2) Complete (nothing missing), 3) In the right order, 4) Has an ending point.

CHAPTER 2 ACTIVITIES

Challenge 1:

Write an algorithm for your morning routine (at least 10 steps)

Challenge 2:

Create an algorithm for finding the largest number in a list

Challenge 3:

Draw a flowchart for deciding what to wear based on weather

CODE THIS! (Pseudocode Exercise)

Write an algorithm that finds a specific word in a list of 20 words.

CHAPTER 2 PUZZLE

SEQUENCE PUZZLE: Put these 8 steps for making a phone call in the correct order.

[Work Space]

CHAPTER 3

Sequences & Order

A sequence in coding means the ORDER in which instructions happen. Order matters a LOT! Think about it: 'Put on socks, then put on shoes' works great. But 'Put on shoes, then put on socks' is impossible! In coding, if you put instructions in the wrong order, the program breaks or does something unexpected. Every program you use runs line by line in sequence.

KEY CONCEPT:

The computer reads code from top to bottom, one line at a time. If step 3 needs information from step 2, step 2 **MUST** come first.

CHAPTER 3 ACTIVITIES

Challenge 1:

Arrange these scrambled steps to make them work correctly

Challenge 2:

Write a 5-step sequence for making hot chocolate

Challenge 3:

Find the error: what happens if step 3 and 4 are swapped?

CODE THIS! (Pseudocode Exercise)

Arrange these steps to bake a cake in the correct SEQUENCE.

CHAPTER 3 PUZZLE

ORDER CHALLENGE: Number these from 1-6 to make a working morning routine.

[Work Space]

CHAPTER 4

Loops: Repeat!

A loop is a set of instructions that repeats over and over. Without loops, you would have to write the same code hundreds of times! Imagine telling someone to 'clap 100 times.' Would you write 'clap' 100 times? No! You would say 'repeat clapping 100 times.' That is a loop. Loops can repeat a set number of times, or keep going until something happens.

KEY CONCEPT:

Types: FOR loops repeat a specific number of times. WHILE loops repeat until a condition changes. INFINITE loops never stop (usually a bug!).

CHAPTER 4 ACTIVITIES

Challenge 1:

Do 10 jumping jacks - you just used a FOR loop!

Challenge 2:

Walk until you reach the wall - you just used a WHILE loop!

Challenge 3:

Write pseudocode using REPEAT to draw 4 sides of a square

CODE THIS! (Pseudocode Exercise)

Write a loop that counts from 1 to 20 and prints each number.

CHAPTER 4 PUZZLE

LOOP PUZZLE: What does this loop print? **REPEAT 5 TIMES:** `print(counter * 2)`

[Work Space]

CHAPTER 5

Conditionals: If/Then

Conditionals let programs make decisions! IF something is true, THEN do this. ELSE do that. Your brain uses conditionals all day: IF it is raining, THEN take an umbrella. IF the light is red, THEN stop, ELSE go. Without conditionals, programs would do the same thing every time, no matter what. Conditionals make programs smart and responsive.

KEY CONCEPT:

IF (condition is true) THEN (do action A) ELSE (do action B). You can also chain: IF...ELSE IF...ELSE for multiple options.

CHAPTER 5 ACTIVITIES

Challenge 1:

Write IF/THEN rules for: What to wear based on temperature

Challenge 2:

Create a yes/no decision tree for choosing a snack

Challenge 3:

Write pseudocode: IF score > 90, print A, ELSE IF > 80, print B...

CODE THIS! (Pseudocode Exercise)

Write IF/THEN logic for a vending machine (insert coin, select item, dispense).

CHAPTER 5 PUZZLE

DECISION TREE: Follow the IF/THEN paths to find what activity to do today.

[Work Space]

CHAPTER 6

Variables: Containers for Info

A variable is like a labeled box that holds information. You give it a name and put data inside. The data can change (that is why it is called a VARIABLE!). Example: `playerScore = 0`. Later: `playerScore = 10`. The box called `playerScore` now holds 10 instead of 0. Variables can hold numbers, text, true/false values, and more.

KEY CONCEPT:

Rules for variables: 1) Give clear names (`playerName`, not `x`). 2) They can change value. 3) Each holds one thing at a time.

CHAPTER 6 ACTIVITIES

Challenge 1:

Make 3 labeled boxes - put different things in each (name, age, fav color)

Challenge 2:

Change a variable: myScore starts at 0, add 5, add 10. What is it now?

Challenge 3:

Write pseudocode using variables to calculate area of a rectangle

CODE THIS! (Pseudocode Exercise)

Create variables to store: your name, age, grade, and favorite subject.

CHAPTER 6 PUZZLE

VARIABLE TRACKER: Follow the code and write what each variable equals after each line.

[Work Space]

CHAPTER 7

Debugging: Finding Mistakes

Debugging means finding and fixing errors in code. The word comes from a real moth found inside a computer in 1947! Every programmer makes mistakes - even experts. The key is learning to find them systematically. Common bugs: typos, wrong order, missing steps, wrong variable names. Good debuggers are patient, logical, and check one thing at a time.

KEY CONCEPT:

Debugging strategy: 1) Read the error message. 2) Find where it broke. 3) Check what changed. 4) Fix ONE thing and test again.

CHAPTER 7 ACTIVITIES

Challenge 1:

Find the 3 bugs in this sandwich algorithm (given scrambled steps)

Challenge 2:

Play 'spot the difference' between working and broken code

Challenge 3:

Write about a time you solved a problem by trying different solutions

CODE THIS! (Pseudocode Exercise)

Here is broken code - find and fix the 3 bugs to make it work.

CHAPTER 7 PUZZLE

BUG HUNT: Find 5 errors in this recipe-algorithm. Circle and fix each one.

[Work Space]

CHAPTER 8

Functions: Mini-Programs

A function is a reusable chunk of code with a name. Instead of writing the same 10 lines over and over, you write them once as a function and call it by name whenever needed. Example: a 'makeBreakfast' function might contain steps for toast, eggs, and juice. Every time you want breakfast, just call `makeBreakfast()` instead of rewriting all steps!

KEY CONCEPT:

Functions: 1) Have a name, 2) May take inputs (ingredients), 3) Do something specific, 4) May give back a result (output).

CHAPTER 8 ACTIVITIES

Challenge 1:

Create a function called 'greetPerson' that takes a name as input

Challenge 2:

List 3 everyday 'functions' (routine tasks you repeat daily)

Challenge 3:

Write a function that calculates the tip at a restaurant

CODE THIS! (Pseudocode Exercise)

Write a function called calculateArea that takes width and height.

CHAPTER 8 PUZZLE

FUNCTION MATCH: Match each function name to what it does (draw lines).

[Work Space]

CHAPTER 9

Input & Output

Programs take INPUT (information going IN), process it, and produce OUTPUT (results coming OUT). When you type your name in a game, that is input. When the game says 'Welcome, [name]!' that is output. A calculator takes number inputs and outputs answers. Understanding input/output helps you design programs that interact with users.

KEY CONCEPT:

INPUT: keyboard typing, mouse clicks, voice, camera, touch screen. OUTPUT: screen display, sound, printing, vibration.

CHAPTER 9 ACTIVITIES

Challenge 1:

List 5 inputs and 5 outputs of your favorite app

Challenge 2:

Design a simple app: what inputs does it need? What does it output?

Challenge 3:

Write pseudocode that asks for name, then prints a greeting

CODE THIS! (Pseudocode Exercise)

Design input/output for a calculator that converts Celsius to Fahrenheit.

CHAPTER 9 PUZZLE

I/O MATCHING: Match each input device to its corresponding output.

[Work Space]

Simple Game Design Concepts

Every video game uses the coding concepts you have learned! Games have: LOOPS (the game keeps running), CONDITIONALS (if player hits enemy, lose life), VARIABLES (score, lives, level), FUNCTIONS (jump, attack, move). Game design starts with an idea, then you plan the rules, characters, and win/lose conditions before writing any code.

KEY CONCEPT:

Game elements: Player, Goal, Obstacles, Rules, Score, Levels, Win/Lose conditions, Game Loop (repeat until game over).

CHAPTER 10 ACTIVITIES

Challenge 1:

Design a simple board game using IF/THEN rules

Challenge 2:

Plan a video game: character, goal, 3 obstacles, scoring system

Challenge 3:

Write pseudocode for a number guessing game

CODE THIS! (Pseudocode Exercise)

Write pseudocode for Rock-Paper-Scissors game against the computer.

CHAPTER 10 PUZZLE

GAME LOGIC: Fill in the missing IF/THEN rules for this simple game.

[Work Space]

Web Basics: HTML Intro

Every website you visit is built with HTML (HyperText Markup Language). HTML uses 'tags' to tell the browser what to show. Tags look like this: `<h1>Big Title</h1>`. The `<h1>` tag means 'make this a big heading.' HTML is not really programming - it is more like formatting - but it is the foundation of the web. Combined with CSS (styling) and JavaScript (behavior), you can build any website!

KEY CONCEPT:

Basic tags: `<h1>` headings, `<p>` paragraphs, `<img>` images, `<a>` links, `<ul><li>` lists. Every tag that opens must close.

CHAPTER 11 ACTIVITIES

Challenge 1:

Write simple HTML for a page about yourself (name, hobbies, picture idea)

Challenge 2:

Look at a website and guess what HTML tags were used

Challenge 3:

Design a webpage layout on paper before coding it

CODE THIS! (Pseudocode Exercise)

Write HTML-style tags to structure a simple webpage about your pet.

CHAPTER 11 PUZZLE

TAG MATCHING: Match each HTML tag to the correct description.

[Work Space]

App Thinking

Apps solve problems for people. Every app started as an IDEA about how to help someone. App thinking means: 1) Find a problem, 2) Think about who has this problem, 3) Design a solution, 4) Plan the features, 5) Build a simple version first, 6) Get feedback and improve. You do not need to know how to code perfectly to THINK like an app developer!

KEY CONCEPT:

Start small: the first version of every app is simple. Facebook started as just profiles. Instagram started as just photo filters.

CHAPTER 12 ACTIVITIES

Challenge 1:

Identify 3 problems at school or home that an app could solve

Challenge 2:

Pick one and design screens on paper (what buttons, what happens)

Challenge 3:

Write the user story: As a [user], I want to [action], so that [benefit]

CODE THIS! (Pseudocode Exercise)

Plan an app: write the features list and draw 3 screen layouts.

CHAPTER 12 PUZZLE

APP DESIGN: Sketch 4 screens for a homework reminder app.

[Work Space]

AI Basics for Kids

Artificial Intelligence (AI) is when computers learn from data instead of just following instructions. Regular programs do exactly what you tell them. AI programs learn patterns and make predictions! When Netflix suggests shows or your phone autocorrects words, that is AI. AI learns from LOTS of examples - like how you learned to recognize cats by seeing thousands of cats.

KEY CONCEPT:

AI needs: 1) Lots of data (examples), 2) A learning algorithm, 3) Computing power. AI is not magic - it is math and patterns!

CHAPTER 13 ACTIVITIES

Challenge 1:

Train a friend: show them 20 drawings and have them guess the pattern

Challenge 2:

List 5 places you encounter AI in daily life

Challenge 3:

Discuss: what can AI do well? What can humans do better?

CODE THIS! (Pseudocode Exercise)

Describe how you would train an AI to recognize dogs vs cats.

CHAPTER 13 PUZZLE

PATTERN FINDER: What comes next in each sequence? (AI thinking!)

[Work Space]

CHAPTER 14

Coding Careers

People who code build the future! Careers include: Software Engineer (builds apps), Game Developer (makes games), Data Scientist (finds patterns in data), Web Developer (builds websites), Cybersecurity Expert (protects systems), Robotics Engineer (programs robots), AI Researcher (teaches computers to learn). All of these are in HIGH demand and pay well!

KEY CONCEPT:

You do not need to decide now! Try different areas and see what excites you. The best coders love solving puzzles and never stop learning.

CHAPTER 14 ACTIVITIES

Challenge 1:

Research 3 coding careers that interest you

Challenge 2:

Interview someone who codes for their job (or watch a video)

Challenge 3:

Set a goal: what coding skill will you learn this month?

CODE THIS! (Pseudocode Exercise)

Write your ideal coding job description - what would you build?

CHAPTER 14 PUZZLE

CAREER QUIZ: Answer 10 questions to discover your ideal coding career path.

[Work Space]

Your Coding Journey Plan

You have learned the fundamentals of computational thinking! Now what? Start with visual coding tools like Scratch, then try Python or JavaScript. Build small projects. Join coding clubs. Watch tutorials. The key is PRACTICE - code a little bit every day. Every expert programmer started exactly where you are right now. Your coding journey is just beginning!

KEY CONCEPT:

Resources: Scratch (scratch.mit.edu), Code.org, Khan Academy, Python for Kids books, YouTube coding tutorials.

CHAPTER 15 ACTIVITIES

Challenge 1:

Create your 30-day coding challenge plan

Challenge 2:

Pick your first language and find a free tutorial

Challenge 3:

Start your first project: something small that excites YOU

CODE THIS! (Pseudocode Exercise)

Create a 4-week learning plan: what will you study each week?

CHAPTER 15 PUZZLE

CODING BINGO: Complete 5 in a row of coding activities this month!

[Work Space]

CODING VOCABULARY

Algorithm:	Step-by-step instructions
Bug:	An error in code
Debug:	Finding and fixing bugs
Loop:	Code that repeats
Variable:	A named container for data
Function:	A reusable block of code
Conditional:	IF/THEN decision making
Input:	Data going into a program
Output:	Results from a program
Syntax:	The grammar rules of a language

FUTURE PROGRAMMER CERTIFICATE

Awarded to:

For completing this entire book!

Date: _____

Author: Daniel Tesfamariam

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

[illegible]

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

[illegible]

Date: _____

Today I learned:

How I will use this:

My goal for tomorrow:

Free writing space:

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Date: _____

Today I learned:

How I will use this:

My goal for tomorrow:

Free writing space:

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

[illegible]

CHALLENGE:

Think of 3 new ideas related to this topic and explain each one.

My Work:

[illegible]

Self-rating (circle): AMAZING / GOOD / NEEDS MORE WORK

CHALLENGE:

Create a mind map connecting everything you learned.

My Work:

[illegible]

Self-rating (circle): AMAZING / GOOD / NEEDS MORE WORK

CHALLENGE:

Write a letter explaining this topic to a younger child.

My Work:

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Self-rating (circle): AMAZING / GOOD / NEEDS MORE WORK

CHALLENGE:

Design a poster that teaches the most important concept.

My Work:

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Self-rating (circle): AMAZING / GOOD / NEEDS MORE WORK

CHALLENGE:

List 10 questions you still have about this topic.

My Work:

[illegible]

Self-rating (circle): AMAZING / GOOD / NEEDS MORE WORK

CHALLENGE:

Draw a comic strip showing what you learned in action.

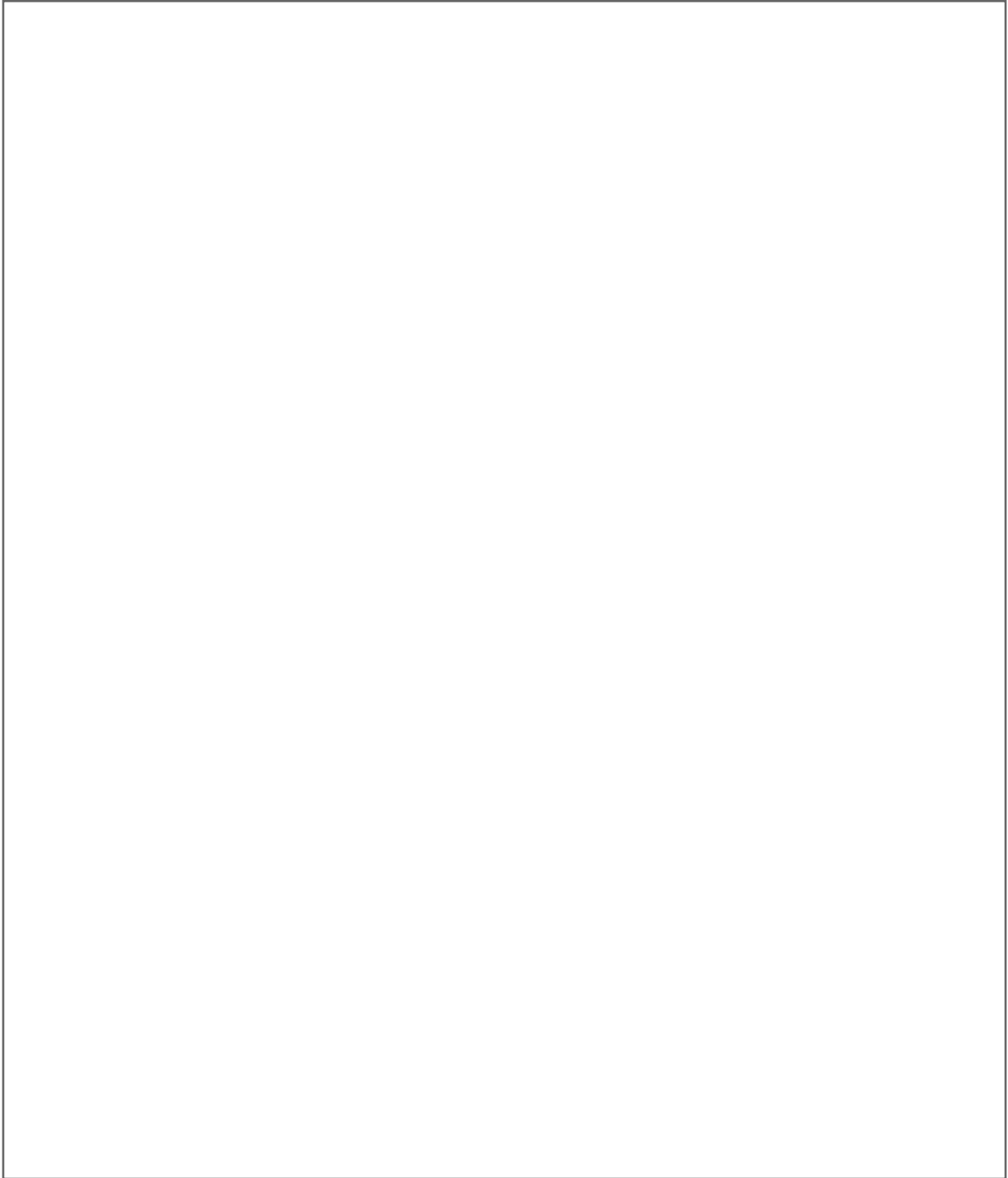
My Work:

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Self-rating (circle): AMAZING / GOOD / NEEDS MORE WORK

CREATIVE SPACE: CODING

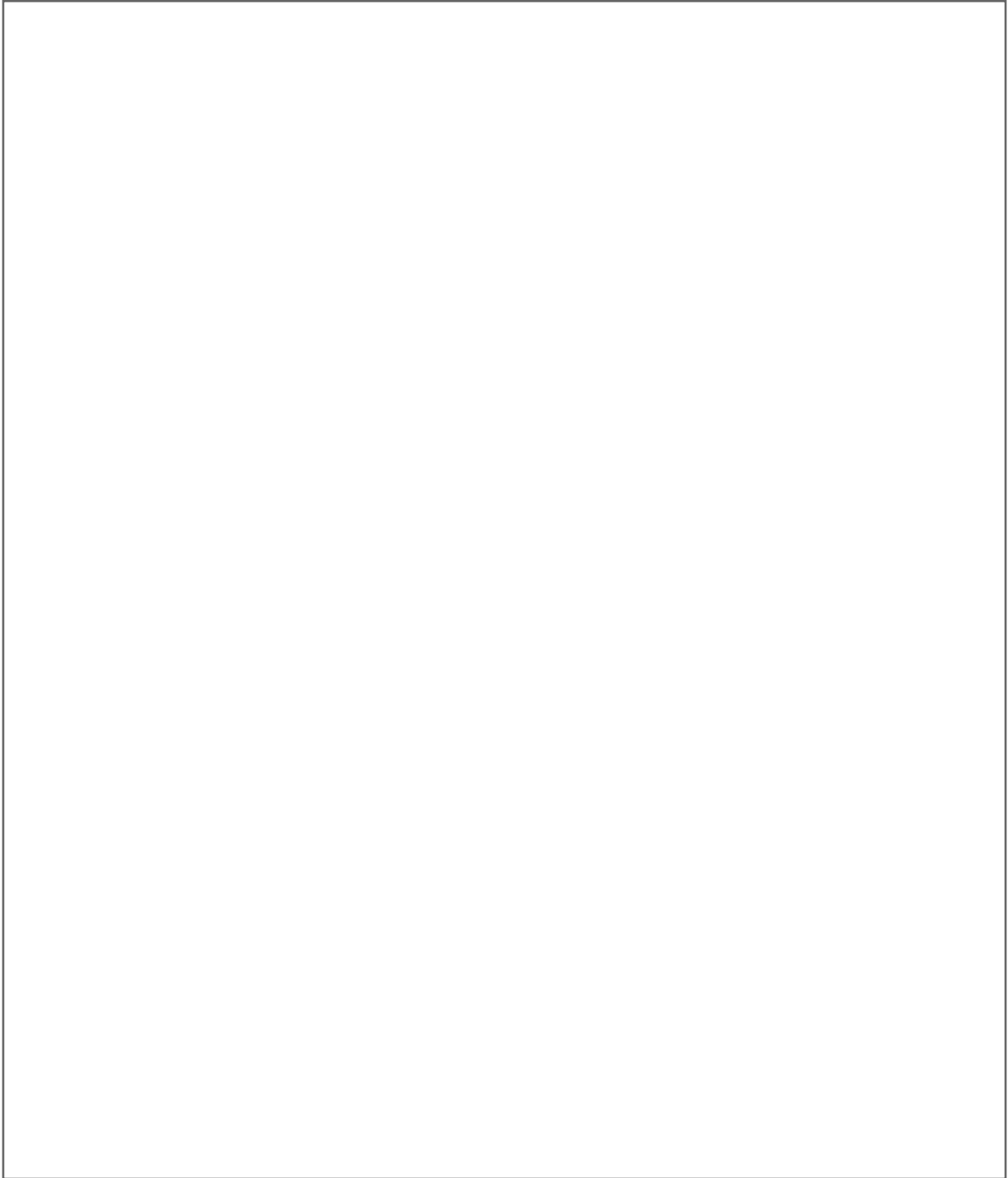
Draw what you learned today

A large, empty rectangular box with a thin black border, intended for a student to draw or write about what they learned in coding.

Use this space for drawing, writing, or brainstorming!

CREATIVE SPACE: CODING

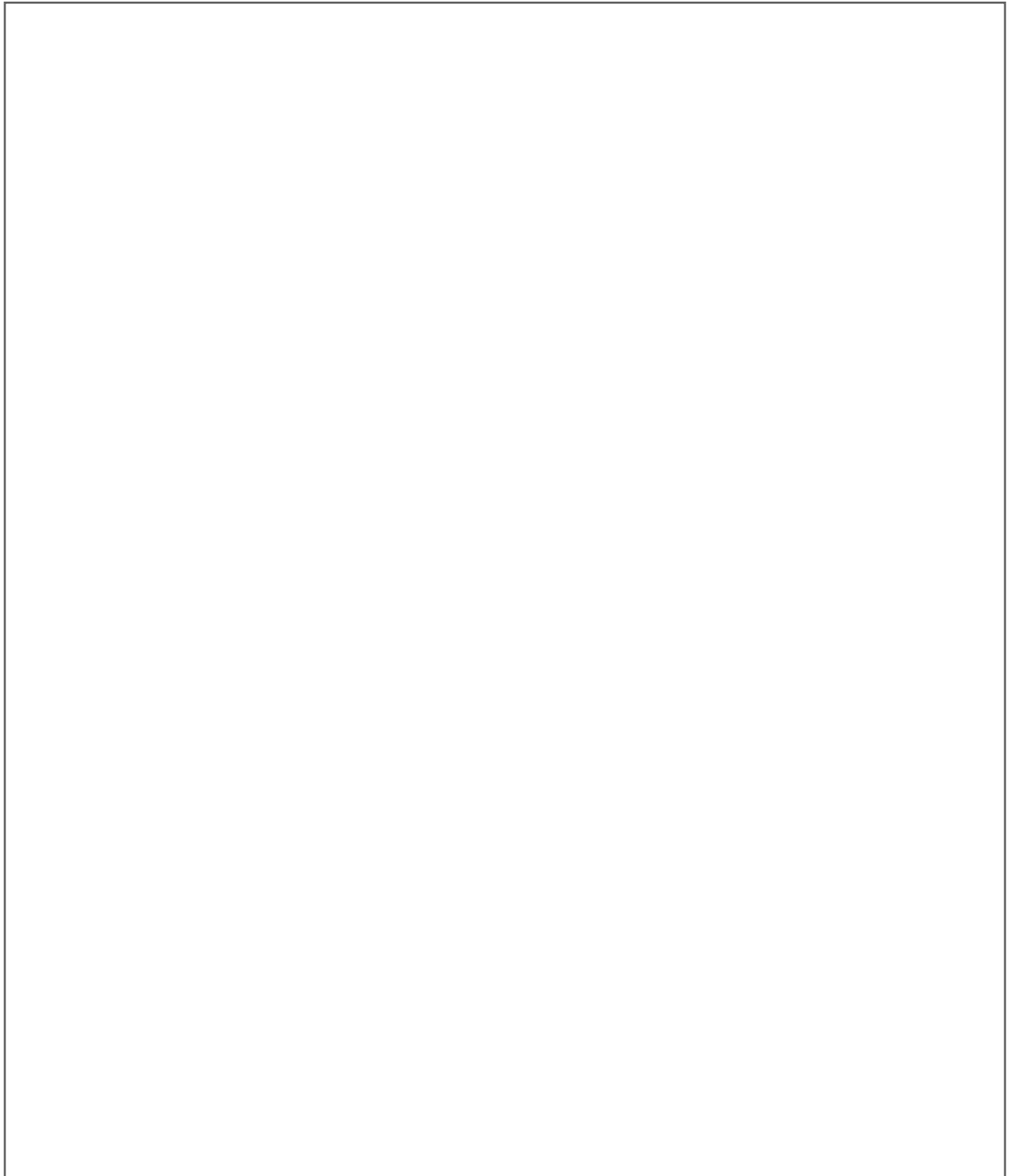
Illustrate your favorite concept

A large, empty rectangular box with a thin black border, intended for drawing, writing, or brainstorming. It occupies the central portion of the page, below the title and instructions.

Use this space for drawing, writing, or brainstorming!

CREATIVE SPACE: CODING

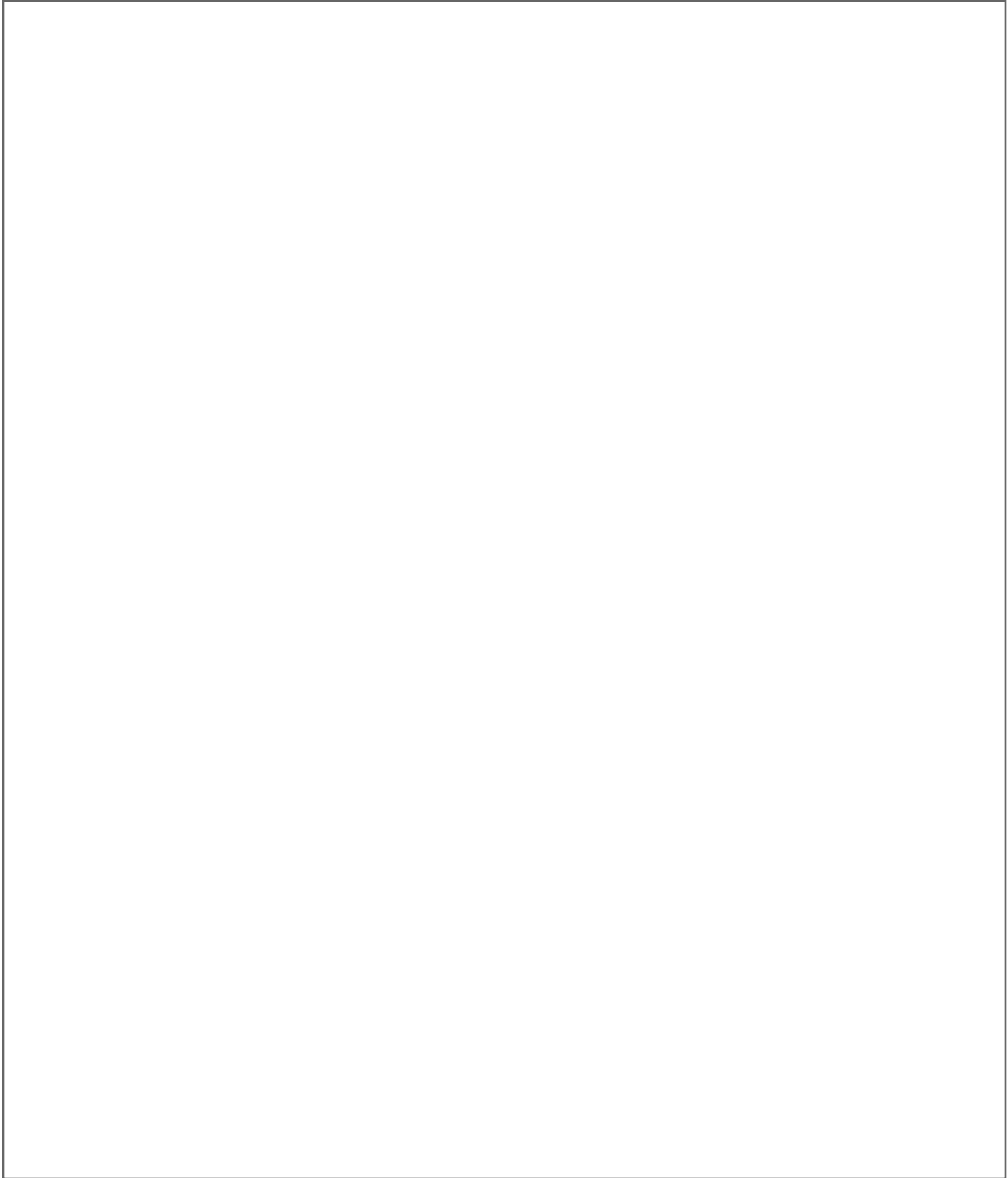
Create a visual summary of this chapter

A large, empty rectangular box with a thin black border, intended for a visual summary or creative work.

Use this space for drawing, writing, or brainstorming!

CREATIVE SPACE: CODING

Draw yourself using this knowledge

A large, empty rectangular box with a thin black border, intended for drawing, writing, or brainstorming. It occupies the central portion of the page, below the introductory text and above the concluding text.

Use this space for drawing, writing, or brainstorming!

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]